

从 8K 到 1M：大语言模型长上下文扩展技术

位置编码、训练方法、注意力系统与能力评测

Jiguo Li¹

jiguolee@gmail.com

调研与修订日期：2026 年 8 月 9 日

摘要

长上下文能力并不能通过简单调大配置文件中的最大长度获得。要实现可靠的长上下文，至少需要满足四项条件：(1) 位置表示在超出训练长度后仍保持数值稳定；(2) 模型通过长序列训练学会检索、整合和推理；(3) 注意力计算、KV 缓存、设备通信和服务调度能够承担长度增长带来的开销；(4) 评测能够区分“可以输入”与“能够有效利用”。本文先从统一的数学视角梳理位置编码，比较绝对位置、相对位置、RoPE 与 ALiBi；再以 8K、16K、32K、256K 和 1M 为工程节点，分析位置插值、NTK-aware 缩放、YaRN、LongRoPE、长文本继续训练、FlashAttention、稀疏与滑窗注意力以及 Ring Attention 的适用范围；最后给出训练、推理、评测和技术选型框架。本文的核心观点是：位置扩展提供新的“坐标系”，长文本训练让模型掌握相应能力，系统优化则保证方案能够实际运行，三者缺一不可。

关键词：大语言模型；长上下文；位置编码；RoPE；位置插值；YaRN；LongRoPE；FlashAttention

阅读提要：位置缩放解决“坐标能否延伸”，长文本训练解决“模型能否使用”，高效注意力与 KV 系统解决“成本能否承受”。标称窗口只有在这三条链路同时成立时，才等价于有效上下文。

¹本文在 Codex 协助下完成

目录

目录

1 问题定义：长上下文究竟扩展了什么	3
2 位置编码技术铺垫	3
2.1 绝对位置：把“第几个”写入表示	3
2.2 相对位置：直接描述 token 间距离	3
2.3 RoPE：以旋转同时表达绝对位置与相对相位	4
3 RoPE 上下文扩展的核心方法	4
3.1 外推、插值与动态缩放	4
3.2 YaRN：频段分治与注意力温度修正	4
3.3 LongRoPE：非均匀搜索与渐进式扩展	4
3.4 从位置扩展到注意力架构改造	5
4 从 8K 到 1M 的分阶段工程路线	5
4.1 8K 到 16K：低倍率迁移	5
4.2 16K 到 32K：频率感知缩放加长文适配	5
4.3 32K 到 256K：训练与系统开始共同主导	5
4.4 256K 到 1M：渐进位置迁移与分布式上下文	6
5 与经典实现协同：从论文公式到可运行配置	6
5.1 Transformers：位置方法的配置落点	6
5.2 Megatron Core：把序列维度作为并行轴	7
5.3 技术阶段与实现栈的对应关系	7
6 训练：把位置可用转化为能力可用	7
6.1 数据与长度课程	7
6.2 可执行的数据构造流水线	8
6.3 SFT 样本如何生成与验收	8
6.4 目标函数与能力塑形	8
6.5 稳定性与短上下文恢复	8
7 推理与服务：真正的百万 token 成本	8
7.1 预填充与解码是两种不同瓶颈	9
7.2 稠密、稀疏与外部记忆的取舍	9
8 评测：避免把容量当能力	9
8.1 由浅入深的四层评测	9
8.2 推荐验收门槛	9
9 方案选型与实施清单	9
10 知名大模型报告所呈现的技术演进	10
10.1 从“扩位置”到“长文再训练”：Llama 3 的 128K 路线	10
10.2 从文本窗口到多模态记忆：Gemini 1.5 的 1M 路线	10
10.3 从模型能力到可部署系统：Qwen2.5-1M	10
10.4 从后处理稀疏到原生稀疏：DeepSeek-V3.2	10
10.5 从长文档到长期智能体：Kimi Linear 与 Kimi K3	11
10.6 演进规律总结	11
11 开放问题与研究展望	11
11.1 有效上下文远小于标称上下文	11
11.2 位置编码仍缺少统一理论	11
11.3 长文推理与检索能力不是同一能力	11
11.4 平方复杂度与信息价值不匹配	12
11.5 KV 缓存将成为服务资源与记忆治理问题	12
11.6 数据稀缺、污染和长尾结构	12
11.7 多模态与 agent 上下文需要新的位置语义	12
11.8 可持续性与经济性	12
12 结论	12

1 问题定义：长上下文究竟扩展了什么

设序列长度为 N 、隐藏维度为 d 。标准自注意力为

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + M\right)V, \quad (1)$$

其中 M 为因果掩码。自注意力本身无法识别 token 的先后顺序，因此必须额外注入位置信息。标准全注意力的计算复杂度为 $O(N^2d)$ ，显式注意力矩阵的空间复杂度为 $O(N^2)$ 。自回归推理虽然可以借助 KV 缓存避免重复计算，但缓存占用仍会随 N 线性增长。

“上下文窗口为 L ”至少可以指四个不同层次：接口能够接收长度为 L 的输入；位置编码在这一范围内不会发生数值发散；模型在这一范围内仍能保持可接受的困惑度；模型能够可靠利用不同位置的信息，并完成跨段推理。讨论长上下文时必须说明具体指哪一层，不能把“接口收得下”直接等同于“模型用得好”。

表 1 长度增长带来的理论规模（仅展示 N^2 ，未计层数与头数）

上下文	N	N^2 元素数	相对 8K 倍数
8K	8,192	6.71 千万	1
16K	16,384	2.68 亿	4
32K	32,768	10.74 亿	16
256K	262,144	687.19 亿	1,024
1M	1,048,576	1.10 万亿	16,384

2 位置编码技术铺垫

2.1 绝对位置：把“第几个”写入表示

原始 Transformer 使用固定正弦/余弦编码 [1]：

$$PE_{(p,2i)} = \sin(p/10000^{2i/d}), \quad (2)$$

$$PE_{(p,2i+1)} = \cos(p/10000^{2i/d}), \quad (3)$$

并令 $h_p = e_p + PE_p$ 。不同维度对应不同波长，因此从公式上看可以计算任意位置。不过，“能够计算”并不意味着模型能够泛化到训练时从未见过的位置。可学习的绝对位置嵌入更直观，但通常受固定表长限制；超出位置表的范围后，编码要么没有定义，要么从未得到训练。

绝对编码把内容和位置相加，便于实现，却不是注意力最直接需要的量。注意力更关心 token 间的相对位移 $m - n$ ，而非孤立的绝对编号。

2.2 相对位置：直接描述 token 间距离

相对位置方法在注意力分数或 value 中加入与 $m - n$ 有关的项。它所具有的平移不变性更符合序列关系，也便于复用历史片段；相应的代价则可能是增加参数量、计算量或实现复杂度。Transformer-XL 把分段循环记忆与相对位置编码结合起来 [24]，对后来的长上下文架构产生了重要影响。

ALiBi 不向 token 表示加入位置向量，而为第 h 个注意力头增加线性距离惩罚 [3]：

$$s_{mn}^{(h)} = \frac{q_m^\top k_n}{\sqrt{d_h}} - a_h(m - n), \quad n \leq m. \quad (4)$$

不同注意力头采用不同的斜率 a_h ，从而形成由局部到长程的多种关注尺度。ALiBi 结构简单，并具备一定的“短序列训练、长序列推理”能力。不过，线性距离惩罚也可能使模型过度偏向近邻信息。对于已经采用 RoPE 的主流开源模型，改用 ALiBi 的成本通常高于直接调整 RoPE。

2.3 RoPE：以旋转同时表达绝对位置与相对相位

RoPE 最初由 *RoFormer: Enhanced Transformer with Rotary Position Embedding* 系统提出 [2]。它将查询和键按二维子空间旋转；对第 i 个频率，令 $\theta_i = b^{-2i/d_h}$ ，旋转矩阵为

$$R(p, \theta_i) = \begin{bmatrix} \cos(p\theta_i) & -\sin(p\theta_i) \\ \sin(p\theta_i) & \cos(p\theta_i) \end{bmatrix}. \quad (5)$$

使用 $q_p = R(p)W_q x_p$ 、 $k_m = R(m)W_k x_m$ 后，

$$q_p^\top k_m = (W_q x_p)^\top R(m - p)(W_k x_m), \quad (6)$$

此时，内积中的位置项只显式依赖相对位移。RoPE 不需要额外的位置表，与注意力机制结合紧密，还可以通过调整位置索引或旋转频率扩展，因此成为长上下文扩展中应用最广的一类位置编码。

RoPE 的主要风险在于频谱失配。高频维度旋转较快，超出训练长度后容易出现模型从未见过的相位组合；低频维度则可能在原训练窗口内尚未经过完整的旋转周期。若对所有频率统一拉伸，局部位置分辨率会下降；若完全不做拉伸，直接外推又可能失稳。

表 2 主要位置表示的工程比较

方法	注入位置	优点	长度扩展局限
固定绝对编码	输入表示	无参数、公式简单	训练外位置仍可能分布外
可学习绝对嵌入	输入表示	表达灵活	固定表长；扩表需训练
相对位置	分数或值	直接建模距离	实现与计算可能更复杂
ALiBi	注意力分数	简洁、可外推	近因偏置可能压制远距信息
RoPE	查询与键	相对关系自然、易缩放	频率和相位在训练外失配

3 RoPE 上下文扩展的核心方法

3.1 外推、插值与动态缩放

设预训练最大长度为 L_0 ，目标长度为 $L_1 = sL_0$ 。朴素外推继续使用位置 $p > L_0$ ，模型直接进入未知相位区域。位置插值 (PI) 改用

$$p' = p/s, \quad (7)$$

将目标区间 $[0, L_1)$ 压回已训练区间 $[0, L_0)$ 。PI 通过少量微调，将采用 RoPE 的 LLaMA 模型扩展到 32K，并表现出优于直接外推的稳定性 [4]。它的代价是所有位置间距都会被同时压缩，相邻 token 的旋转角差也会随之减小，因而可能损害局部位置分辨率。

NTK-aware 缩放可以看作一类频率重参数化方法。它不再等比例压缩所有维度，而是通过调整 RoPE 的基频或各维频率，尽量让高频维度保留局部模式，同时由低频维度承载更长距离的信息。动态缩放则根据实际输入长度确定缩放幅度，以减轻短输入上的性能下降。由于社区中的具体公式并不完全一致，工程文档必须记录基频、缩放因子和实现版本，不能只笼统地写成“使用 NTK”。

3.2 YaRN：频段分治与注意力温度修正

YaRN 根据各频率维度在原训练窗口内经历的旋转周期进行分组：高频维度已经得到充分训练，更适合采用插值；低频维度尚未充分旋转，更适合保持原状；中间频段则采用平滑过渡。与此同时，YaRN 还通过 attention scaling 修正上下文扩展后注意力熵的变化。论文表明，与已有方法相比，YaRN 可以显著减少微调所需的 token 数和训练步数，并把上下文扩展到 128K [5]。

YaRN 的价值不在于某个特殊常数，而在于它明确考虑了 RoPE 频谱的非均匀性：对所有维度采用相同的处理方式通常并非最优。即使如此，模型一般仍需要接受长文本微调，才能适应新的距离分布。

3.3 LongRoPE：非均匀搜索与渐进式扩展

LongRoPE 进一步利用了两类非均匀性：不同 RoPE 维度可以采用不同的缩放因子，序列开头的一段 token 也可以单独处理。该方法先通过搜索确定插值参数，把模型微调到 256K，然后对扩展后的模型进行第二次位置插值。论

文在 LLaMA2 和 Mistral 上报告了最高 2,048K 的上下文长度, 并通过短上下文重校准恢复模型在原窗口内的能力 [6]。

这表明, 扩展到 1M 更适合采用分阶段迁移, 而不是一次性把 8K 的位置区间压缩 128 倍。论文结果证明了技术上的可行性, 但并不意味着模型在所有任务和所有位置上都同样可靠。

3.4 从位置扩展到注意力架构改造

PI、YaRN 和 LongRoPE 主要解决“位置坐标如何延伸”的问题, 却没有改变稠密注意力的平方复杂度。上下文进入百万 token 后, 研究重点正在从“把 RoPE 拉长”转向“从训练开始就采用更适合长序列的注意力”。这一变化并不意味着位置编码不再重要, 而是表明位置方法必须与注意力结构共同设计。

Native Sparse Attention (NSA) 采用三条互补分支: 压缩分支用块级表示保留全局概貌, 选择分支为每个 query 动态挑选重要 token 或 token 块, 滑动窗口分支则保留精确的局部信息 [19]。与推理时临时裁剪 KV 的方法不同, NSA 从预训练阶段就使用稀疏结构, 使未被选中的连接不再承担前向和反向计算。其设计还特别考虑 GPU 的算术强度和内存访问模式, 说明稀疏注意力是否有效, 既取决于稀疏率, 也取决于稀疏模式能否映射为高效算子。

另一条路线是线性注意力。Kimi Linear 提出 Kimi Delta Attention (KDA), 用带细粒度门控的 delta rule 更新固定大小的状态, 并按约 3:1 的比例交替使用 KDA 与全局 MLA 层 [20]。其中, 线性注意力层负责低成本累积长期状态, 全局注意力层则补充精确的信息回看能力。该工作报告, 在百万 token 解码场景下, 混合架构可显著降低 KV 缓存, 并提高解码吞吐。与稀疏注意力相比, 线性注意力不再显式保留所有历史 token 的 KV, 代价是历史信息被压缩进有限状态, 精确检索能力更依赖状态更新规则和全局层的配置。

因此, 面向百万上下文可以区分三条技术路线:

1. **扩展稠密注意力**: 保留原有模型结构, 使用 RoPE 缩放、继续训练和上下文并行, 质量风险较低, 但成本最高;
2. **原生稀疏注意力**: 只保留局部、压缩或动态选出的连接, 在训练和推理阶段同时降低计算量, 但需要专门训练和硬件友好的稀疏算子;
3. **混合线性注意力**: 多数层维护固定状态, 少数层执行全局注意力, 在 KV 成本上更有优势, 但通常难以从既有稠密模型直接无损迁移。

这三条路线的选择取决于项目目标: 若要低成本改造已有 checkpoint, 仍应优先选择 RoPE 扩展; 若准备从头预训练新模型, 并把百万上下文作为核心能力, 则原生稀疏或混合线性架构更值得考虑。

4 从 8K 到 1M 的分阶段工程路线

4.1 8K 到 16K: 低倍率迁移

扩展倍率只有 $2\times$ 时, 通常可以采用线性 PI、基频调整或动态 RoPE scaling 作为初始方案, 再使用长短混合数据进行一段时间的继续训练。训练批次中必须保留足够的短文本, 否则模型扩展后可能在短文本任务上退化。在这一阶段, FlashAttention 配合梯度检查点通常已经能够控制显存占用。

4.2 16K 到 32K: 频率感知缩放加长文适配

随着扩展倍率提高, 统一 PI 对局部位置的压缩会更加明显。此时宜采用频率感知方法, 如 NTK-aware 或 YaRN, 并加入真实长文、跨段检索和多证据合成样本。PI 论文表明, 在能力较强的预训练模型上, 只需少量微调即可把 LLaMA 系列扩展到 32K [4]; 不过, 论文中的训练步数并不是可以直接套用到所有模型上的固定值。

4.3 32K 到 256K: 训练与系统开始共同主导

此时仅修改位置参数不够。推荐渐进式长度课程 (如 $32K \rightarrow 64K \rightarrow 128K \rightarrow 256K$)、长度分桶和短长样本混合。数据应覆盖长文结构、跨章节依赖、重复干扰项与远距离证据组合。

在系统层面, 需要引入显存友好的精确注意力、序列并行或上下文并行。FlashAttention 采用分块计算和在线 softmax, 避免把完整的 $N \times N$ 注意力矩阵写入高带宽显存, 从而减少内存读写和中间结果占用 [7]; FlashAttention-2 又通过改进线程块和 warp 的任务划分进一步提高了硬件利用率 [27]。不过, 这类精确注意力的计算复杂度仍然是 $O(N^2)$ 。Longformer、BigBird 和 LongNet 分别以局部与全局注意力、块稀疏连接和膨胀注意力降低长序列成本 [25, 26, 29]。滑窗或块稀疏注意力虽然可把计算量降至近似 $O(Nw)$, 但也会改变信息传播路径, 因此通常需要用全局 token、跨层传播或专门的检索机制加以补偿。

4.4 256K 到 1M：渐进位置迁移与分布式上下文

从 8K 到 1M 是 $128\times$ 的坐标扩张，也是 $16,384\times$ 的注意力矩阵规模扩张。可行路线通常包含：

1. 非均匀频率缩放或搜索，避免对所有 RoPE 维度等比例压缩；
2. 先训练到 128K/256K，再做二次插值或继续扩展；
3. 使用序列/上下文并行，把 token 维度分散到多个设备；
4. 采用分块注意力，并尽量让通信与计算重叠。Ring Attention 沿设备组成的环传递 KV 块，可处理的序列长度大致随设备数量成比例增加 [8]；
5. 管理巨大的 KV 缓存：采用 GQA/MQA、低精度或量化 KV、分页缓存、前缀复用，必要时使用滑窗/分层记忆；
6. 以 1M 级合成长检索与真实长文做压力测试，同时保留短上下文回归集。

阶段结论：8K–16K 主要是位置适配问题；16K–32K 开始要求频率感知缩放和长文训练；32K–256K 进入训练系统协同；256K–1M 则是渐进位置迁移、分布式注意力、KV 管理和严格评测的组合工程。

表 3 推荐的分阶段技术组合

目标	位置策略	训练重点	系统重点	主要风险
16K	PI/动态缩放	少量继续训练；短长混合	FlashAttention；检查点	局部分辨率下降
32K	NTK-aware/YaRN 类	跨段检索与合成	高效精确注意力	中段或末段失效
256K	非均匀缩放；渐进扩展	长度课程；真实长文	上下文并行；稀疏选项	成本激增；短文退化
1M	LongRoPE 类二阶段扩展	256K 内训练加超长评测	Ring/块式计算；KV 优化	“能输入但不会用”

5 与经典实现协同：从论文公式到可运行配置

5.1 Transformers：位置方法的配置落点

Hugging Face Transformers 已经把多种 RoPE 方案纳入统一的模型配置。官方工具目前支持 `default`、`linear`、`dynamic`、`yarn`、`longrope` 和 `llama3` 等类型 [9]。这样一来，所选算法可以明确记录在 checkpoint 的元数据中，不必隐藏在训练脚本里。典型配置大致如下：

```
config.rope_parameters = {
    "rope_type": "yarn",          # 或 dynamic / longrope / llama3
    "factor": 8.0,
    "original_max_position_embeddings": 8192,
    "attention_factor": ...
}
```

不同类型所需的配置字段并不相同；LongRoPE 还可能包含逐维设置的短序列因子和长序列因子。因此，在迁移 checkpoint 时，应同时保存 Transformers 版本、模型配置和实际采用的 attention 实现。单独调大最大位置长度，既不会自动改变 RoPE 频谱，也不会让模型凭空获得长文本能力。

Transformers 适合用于验证位置方案、加载公开 checkpoint，以及开展中小规模的 CPT 或 SFT。训练时还应明确选择 `flash_attention_2` 或 SDPA，启用 gradient checkpointing，并按长度对样本分组，以减少 padding 带来的浪费。LongLoRA 进一步表明，可以将移位稀疏注意力与参数高效微调结合，在训练阶段降低长序列成本，而推理阶段仍使用稠密注意力 [30]。面对超长样本，仅靠单卡 Trainer 通常难以满足需求，还需要结合 DeepSpeed、FSDP 或 Megatron 的上下文并行能力。

5.2 Megatron Core: 把序列维度作为并行轴

经典 Megatron-LM 将张量并行 (TP)、流水线并行 (PP) 和数据并行 (DP) 组合用于大模型训练 [10]。长上下文进一步引入上下文并行 (CP): 沿序列维度切分输入和激活, 每张 GPU 只常驻约 N/C 个 token 的激活; 注意力阶段再交换 KV 块。Megatron Core 官方说明 CP 与 Ring Attention 相似, 并通过环形点对点通信、FlashAttention 内核及因果负载均衡减少额外开销 [11]。

一个 128K 训练任务的配置思路可写为:

```
--seq-length 131072
--context-parallel-size 8
--cp-comm-type p2p
--tensor-model-parallel-size 4
--pipeline-model-parallel-size 4
--sequence-parallel
--use-flash-attn
```

其中, CP 沿序列长度切分计算, TP 切分层内矩阵, PP 切分网络层, DP 则切分训练批次。这里尤其需要区分 **sequence parallelism** 和 **context parallelism**: 前者主要把 LayerNorm、Dropout 等注意力之外的激活沿序列维度分摊到 TP rank; 后者会切分全部网络输入和激活, 并在 attention 计算时交换完整序列所需的 KV[11]。对于采用 GQA 或 MQA 的模型, 由于 KV 头更少, CP 的通信量也会相应降低 [28]。

在因果 attention 中, 序列前半段 token 的计算量较小, 后半段则较大, 因此简单地连续切分序列会造成 GPU 负载不均。实际实现中常采用 zigzag 或双端分块, 把序列首尾的块配对分配。CP 的规模还应结合序列长度、节点拓扑和通信带宽确定; 如果一味增大 CP, 通信反而可能成为主要瓶颈。

5.3 技术阶段与实现栈的对应关系

表 4 从研究方法到经典软件栈的映射

长度	算法层	Transformers 落点	Megatron/系统落点
8K–16K	linear/dynamic RoPE	rope_parameters; FlashAttention/SDPA	TP/DP; activation checkpointing
16K–32K	YaRN/频率感知缩放	rope_type=yarn; 长度分组	SP 降低非 attention 激活; 可启用 CP
32K–256K	非均匀缩放 + CPT	LongRoPE/llama3 配置; packed dataset	CP + TP + PP; FlashAttention; zigzag 分块
256K–1M	渐进扩展 + 稀疏/分布式 attention	推理端 chunked prefill; 模型专用实现	分层 CP/Ring; KV 分页、量化与流水线

这张表说明了算法与框架各自承担的职责: Transformers 主要定义位置函数和模型行为, Megatron Core 则负责把超长序列的一次前向与反向传播分配到多台设备上。两者可以借助相同的模型检查点格式或转换脚本配合使用, 但必须逐项核对 RoPE 配置、QKV 排列方式、GQA 头数和分词器, 确保彼此一致。

6 训练: 把位置可用转化为能力可用

6.1 数据与长度课程

随机拼接短文只能增加序列长度, 并不会自动形成长距离依赖。有效的长文本数据至少应包括: 完整的单文档叙事、同一主题下的多文档集合、跨章节问答、远距离实体一致性、多跳证据, 以及带有干扰项的检索任务。数据记录中应标明“有效依赖距离”, 不能只记录样本的 token 数。相关研究表明, 将模型扩展到 128K 时, 约 0.5B–5B 个高质量继续预训练 token 就可能取得效果; 但领域平衡和长文本上采样同样重要, 单纯增加书籍类语料并不是最佳做法 [12]。

按阶段逐步增加序列长度, 可以降低训练早期的成本。不过, 如果模型只在最后少量步骤中接触最长序列, 它可能只是适应了新的位置范围, 却没有真正学会利用远距离信息。较稳妥的做法是保留大部分中短样本以维持基础能力, 再逐步提高长样本比例, 并保证每个目标长度都获得足够的训练 token。进行样本打包时, 还应防止不同文档之间发生无意的交叉注意, 必要时显式使用文档边界掩码。

6.2 可执行的数据构造流水线

从原始语料到最终训练批次，可以按以下八个步骤处理：

1. **采集与解析**：保留文档层级、章节标题、页码、代码路径和时间戳；PDF/OCR 文本需去页眉页脚、断行与重复页。
2. **质量过滤**：语言识别、字符异常率、模板占比、困惑度/质量分类器、版权与隐私过滤；代码语料保留仓库结构。
3. **文档级去重**：除对段落使用 MinHash 外，还要对相邻章节和整篇文档进行近重复检测，避免同一本书或同一份报告的多个版本在长样本中占比过高。
4. **长度标注**：用最终 tokenizer 计算 token 数，并记录领域、文档完整性和最大潜在依赖距离；不能用字符数代替 token 数。
5. **分桶与采样**：例如设 $B_0 = [0, 8K)$ 、 $B_1 = [8K, 32K)$ 、 $B_2 = [32K, 128K)$ 、 $B_3 = [128K, 256K]$ 。按“领域 × 长度桶”联合采样，避免最长桶只剩小说或代码。
6. **切分与拼接**：优先保留文档原有的自然边界。对于超长文档，可按章节切成带重叠的窗口；拼接短文时插入 EOS，并通过 document ids 或 varlen attention 阻断跨文档注意力。
7. **长依赖合成**：在不同深度插入唯一键值、多键聚合、变量替换链或冲突证据；保存证据位置，便于按相对深度评测。干扰项在词面和主题上应接近正确证据。
8. **污染检查**：按文档与 n-gram 对 LongBench、RULER 及业务测试集去污染；拆分应先按文档/仓库，再生成窗口，避免同源片段跨 train/test。

下面给出一个可复现的 CPT 起始配比，但它并非固定配方：短文本占 35%，8K–32K 占 25%，32K–128K 占 25%，128K–256K 占 15%。每个长度桶内部都应保持网页、书籍、论文、代码和对话等领域的平衡。训练可以分阶段把最大长度从 32K 提高到 128K，再提高到 256K；各阶段应按 token 预算而不是样本数量计量。如果同一 batch 中的长度差异较大，可先按长度排序分桶，再进行 packing。LongAlign 的实验表明，排序组批、样本打包和按序列加权损失有助于提高变长 SFT 的训练效率，并平衡不同长度样本对损失的贡献 [13]。

6.3 SFT 样本如何生成与验收

长上下文 SFT 应覆盖“找、合、算、写、引”五类能力，即定位单一证据、整合多处证据、统计表格或日志、生成前后一致的长篇内容，以及给出可核查的引用。可以先从文档中自动抽取实体、数值和章节关系，再让教师模型生成问题与答案，最后使用确定性程序校验证据位置、数值答案和引用范围。若只依赖教师模型自评，容易把模型幻觉带入训练集。

对多文档任务，先选一个主题簇，指定 2–5 个必要证据分散在不同文档和深度，再加入语义相似的冲突/无关文档。答案应附证据 ID；过滤掉无需阅读上下文即可凭常识回答的样本。对代码任务，可从真实仓库的调用图构造跨文件符号追踪、配置传播和 bug 定位问题，并按仓库划分数据集。

设计损失函数时，需要明确是否对提示部分的 token 计算损失。常见的 SFT 只计算 assistant 输出部分的损失；但这样一来，长输入中的关键证据不会直接受到 token 级监督，模型对长文本能力的学习主要依赖输出端传回的梯度。可以增加证据定位或引用生成等辅助目标，也可以按序列归一化不同样本的损失，避免一个 100K 样本仅仅因为 token 更多，就在训练中压过大量短样本。

6.4 目标函数与能力塑形

纯下一 token 预测对长距离证据的梯度可能很弱。可叠加监督任务：在随机深度插入 key-value，要求模型恢复；把互补证据放在远端，要求联合回答；对长摘要设置事实覆盖与引用定位；对代码库设置跨文件调用追踪。合成任务适合控制距离和干扰强度，真实任务负责生态有效性，两者不可互相替代。

6.5 稳定性与短上下文恢复

扩展后常见问题包括注意力熵变化、边界位置性能下降、局部语法能力轻微退化。对策包括注意力温度修正、短长混合训练、分阶段学习率、短上下文重新校准，以及对原始基准做持续回归。LongRoPE 的短窗口 readjustment 体现了这种“双尺度校准”思想 [6]。

7 推理与服务：真正的百万 token 成本

7.1 预填充与解码是两种不同瓶颈

预填充阶段需要一次处理整段输入，全注意力的计算量随 N^2 增长；解码阶段每生成一个 token 都要读取历史 KV，因此通常更受显存带宽限制。FlashAttention 主要减少预填充阶段的内存读写和中间结果占用；GQA、MQA、KV 量化和分页缓存则主要用于降低解码阶段的缓存压力。vLLM 的 PagedAttention 借鉴操作系统分页机制管理非连续 KV 块，可以减少显存碎片并支持跨请求共享前缀 [31]。

对层数 L 、每 token 每层 KV 元素约 $2n_{kv}d_h$ 的模型，KV 缓存字节数近似为

$$\text{KVBytes} \approx N \cdot L \cdot 2n_{kv}d_h \cdot b, \quad (8)$$

其中 b 是每元素字节数。上下文从 256K 到 1M，KV 缓存约再增四倍；即使注意力矩阵不落显存，缓存也可能成为容量瓶颈。

7.2 稠密、稀疏与外部记忆的取舍

稠密注意力允许任意两个 token 直接交互，能力上限较高，但计算成本也最高。滑窗注意力的成本更容易控制，适合以局部依赖为主的文本；由于远距离信息需要跨层逐步传递，精确检索能力可能受到影响。块稀疏注意力和全局 token 提供了折中方案。检索增强生成 (RAG) 则先在模型外筛选信息，成本较低、来源也更容易追踪，但一旦召回阶段出错，关键证据会在送入模型之前就被丢弃。

长上下文与 RAG 并非替代关系：百万窗口适合对已知候选集合进行整体理解，RAG 适合从巨大语料中缩小候选集合。工程上常采用“检索粗筛 + 长上下文精读 + 引用校验”。对于连续生成或长期会话，StreamingLLM 通过保留 attention sink 与近期窗口维持稳定生成，展示了“在固定缓存预算下持续运行”的另一种思路 [32]。

8 评测：避免把容量当能力

8.1 由浅入深的四层评测

1. 数值稳定：各长度困惑度、NaN/溢出、注意力熵、位置分段损失；
2. 单点检索：needle-in-a-haystack 或 passkey，横轴为长度、纵轴为插入深度；
3. 多证据能力：多个远距证据的合取、冲突消解、时间线重建；
4. 真实任务：长文问答、代码库理解、法律/科研文档分析，并检查引用正确性。

单点检索得分高，只能说明模型能够找到那根“针”，并不能证明它理解了整片“草堆”。“Lost in the Middle”研究发现，即使输入没有超过模型的标称上限，当关键信息位于序列中部时，模型表现也可能明显弱于信息位于开头或结尾时 [33]。因此，完整的评测矩阵应覆盖序列开头、中间和末尾的不同位置，以及不同长度和不同干扰强度；同时还要检查原有短上下文能力，并记录吞吐量、首 token 延迟、峰值显存和单次请求成本。

近年的评测已经从合成检索进一步转向真实推理。 ∞ Bench 将中英文长上下文评测推进到平均 100K token 以上 [35]；LongBench v2 又加入单文档问答、多文档问答、长程上下文学习、长对话历史、代码仓库理解和结构化数据理解等任务，原始材料长度从 8K 延伸到约 2M words [21]。这些实验表明，增加推理阶段的计算量可能改善长上下文表现，但长推理并不能替代可靠的信息定位。因此，评测时应把“找对证据”和“根据证据推理正确”分别计分。

建议采用三维评测矩阵：第一维是输入长度和证据相对位置，第二维是任务复杂度（单点、多点、多跳、聚合），第三维是系统预算（稠密、不同稀疏率、不同 KV 容量）。只有这样，才能回答性能下降究竟来自位置外推、注意力路由、推理能力，还是资源预算不足。

8.2 推荐验收门槛

不应设置脱离业务的统一分数。可用相对门槛：目标长度上的关键任务不低于短窗口基线的既定比例；不同位置分桶方差受控；原始短任务退化不超过容忍值；在目标硬件上满足延迟与成本预算。任何“1M 支持”声明都应附带模型版本、位置参数、注意力实现、硬件规模、数据类型、任务和位置热图。

9 方案选型与实施清单

若目标是把现有 RoPE 模型从 8K 扩到 32K，优先采用成熟的 YaRN/动态 NTK 类实现，配合少量长文继续训练和完整回归。若目标是 128K–256K，应把训练数据、上下文并行和 KV 服务设计纳入同一项目。若目标是 1M，应先证明业务确有“整体读入”的收益，并预留分布式预填充、缓存分层和渐进训练预算；否则 RAG 或分层摘要可能

更经济。

表 5 决策矩阵

场景	首选	原因
已有 8K RoPE, 低成本 扩到 32K	频率感知缩放 + 短程微调	架构改动小, 生态成熟
128K 文档分析	YaRN/非均匀缩放 + 长文训练 + FlashAttention	同时处理位置、能力和显存
256K–1M 单请求	渐进扩展 + 上下文并行/Ring + KV 优化	单设备与一次性插值通常不足
超大知识库问答	RAG + 较长精读窗口	先减少无关 token, 成本更可控
局部长程混合 (代 码/视频)	滑窗或块稀疏 + 全局通路	利用结构先验降低算力量

实施过程中应对以下信息进行版本化管理：RoPE 基频与各维缩放因子、最大训练长度和推理长度、attention scaling、训练数据的长度分布、mask 规则、注意力内核、KV 数据类型、并行拓扑以及评测热图。缺少这些信息，实验结果通常很难复现。

10 知名大模型报告所呈现的技术演进

10.1 从“扩位置”到“长文再训练”：Llama 3 的 128K 路线

Llama 3 系列把公开模型的上下文长度提高到 128K[14]。它的意义不只是把窗口数字做大，更在于把长上下文纳入预训练和后训练的完整流程：RoPE 及其频率参数提供位置基础，长序列继续预训练让模型适应新的长度分布，后训练则进一步培养长文问答、总结和工具使用能力。Llama 3 在架构上采用 GQA，也有助于降低长序列解码时的 KV 缓存占用和上下文并行通信量。

Transformers 为 Llama 3 提供了专门的 `rope_type=llama3` 配置 [9]。这说明在实际工程中，已经不能再把各种频率缩放方案笼统地归为“NTK”。使用模型时，应直接读取 checkpoint 自带的高低频缩放参数，而不是只根据目标长度自行设定一个 RoPE base。

10.2 从文本窗口到多模态记忆：Gemini 1.5 的 1M 路线

Gemini 1.5 技术报告展示了最高达到百万 token 的多模态上下文，并把长视频、音频和文本统一为长序列建模问题 [15]。虽然公开信息还不足以复现完整的训练系统，但技术方向已经十分明确：百万级窗口不只是为了容纳更长的文章，还要作为长视频、代码库和多模态时间序列的统一记忆空间。该报告同时采用长上下文检索和多模态任务进行评测，也推动业界从单一文本的 NIAH 测试转向跨模态、跨时间的依赖建模。

Gemini 案例也提醒：闭源模型的窗口声明可以作为能力里程碑，不能作为算法归因证据。若报告没有披露位置编码、稀疏模式或并行策略，应明确标注“未公开”，避免把社区猜测写成事实。

10.3 从模型能力到可部署系统：Qwen2.5-1M

Qwen2.5-1M 是少数较完整公开百万上下文训练和推理链路的技术报告之一 [16]。其方案包括长数据合成、渐进式预训练、多阶段 SFT，以及无需额外训练的长度外推；推理侧则采用稀疏注意力、稀疏精度修正、分块预填充，并对算子、流水线和调度进行优化。报告显示，在 1M 上下文的预填充场景中，整体速度可提高约 3–7 倍。这一案例与本文的三层框架相对应：位置外推保证坐标可用，长上下文预训练和后训练保证能力，稀疏与分块计算则控制系统成本。

这一案例还表明，进入 1M 上下文阶段后，仅发布模型权重已经不够，还应提供配套的推理框架，或至少明确给出可复现的注意力计算规则。如果推理时采用的稀疏模式与训练阶段不同，就可能产生分布偏移，因此需要进行稀疏模式校准，或借助稠密注意力模型进行校正。

10.4 从后处理稀疏到原生稀疏：DeepSeek-V3.2

DeepSeek-V3.2 把稀疏注意力进一步推进到模型训练本身。其 DeepSeek Sparse Attention (DSA) 通过轻量索引器为每个 query 选择相关的历史 token，在长上下文中降低注意力计算量，同时尽量保持模型能力 [22]。与“先训练稠密模型、部署时再裁剪 KV”相比，DSA 代表了一种更深的架构变化：路由器、稀疏模式和主模型在训练阶段共同适应。

这一演进也带来新的工程约束。首先，索引器的召回错误会直接变成模型不可见的信息；其次，动态稀疏的索引和数据搬运可能抵消理论 FLOPs 收益；再次，训练阶段需要保证未被选中 token 仍能获得足够学习信号。因而，稀疏注意力的评测不能只报告平均加速比，还应报告不同证据深度下的召回率、精度损失和尾延迟。

10.5 从长文档到长期智能体：Kimi Linear 与 Kimi K3

Kimi Linear 展示了混合线性注意力在百万上下文中的潜力：KDA 层用固定状态吸收历史信息，全局 MLA 层周期性提供精确回看 [20]。服务系统方面，Mooncake 将预填充与解码解耦，并围绕 KV 缓存的传输和复用组织资源，体现了长上下文服务由“模型中心”转向“KV 中心”的趋势 [36]。到 2026 年发布的 Kimi K3，技术报告进一步把 1M 上下文与原生多模态、长程智能体强化学习及 KDA 的系统协同结合起来 [23]。这说明长上下文的应用对象正在从静态文档转向持续生成的智能体轨迹，包括网页状态、工具调用结果、代码修改和多模态观察。

不过，智能体的长期记忆不宜完全等同于原始上下文窗口。持续任务会产生大量重复、过时或相互冲突的状态。如果不进行选择写入、压缩和遗忘，即使模型拥有 1M 窗口，也可能因噪声增加而降低决策质量。未来的系统更可能采用“短期精确上下文 + 长期压缩状态 + 外部可检索记忆”的分层结构。

10.6 演进规律总结

表 6 代表性模型报告与长上下文技术信号

模型/工作	长度	公开的关键线索	对技术演进的含义
PI/YaRN	32K/128K	RoPE 插值、频率分治、轻量微调	位置扩展成为可迁移模块
Llama 3	128K	GQA、长上下文模型、专用 RoPE 配置	位置、CPT 与后训练协同
LongRoPE	2,048K	非均匀搜索、256K 训练、二次插值	百万级需要渐进迁移
Gemini 1.5	1M	超长多模态输入与检索评测	长上下文成为统一多模态记忆
Qwen2.5-1M	1M	渐进预训练、多阶段 SFT、稀疏 attention、chunked prefill	发布从模型权重扩展到端到端系统
DeepSeek-V3.2	长上下文	原生动态稀疏注意力与索引器	稀疏模式进入预训练架构
Kimi Linear/K3	1M	KDA 与全局 MLA 混合；智能体长轨迹	从 KV 全量保存转向状态化记忆

总体来看，早期研究主要解决“超出训练位置后不崩溃”的问题，随后转向“用少量长文本数据学会远距离检索”，如今则进一步关注百万 token 推理能否真正部署。与此同时，长上下文所处理的对象也从纯文本扩展到代码库、视频、音频和智能体运行轨迹，其作用逐渐由“读取长文档”转向“承载持续更新的工作记忆”。

11 开放问题与研究展望

11.1 有效上下文远小于标称上下文

RULER 的实验表明，许多长上下文模型虽然能在简单的 NIAH 测试中取得较好成绩，但随着序列变长、任务变复杂，性能仍会明显下降；不少标称支持 32K 以上上下文的模型，实际上无法在 32K 长度下保持令人满意的表现 [17]。未来的评测需要从单点检索扩展到多跳推理、信息聚合、反事实判断和长链因果分析，并为“有效上下文长度”建立统计定义和置信区间。

11.2 位置编码仍缺少统一理论

现有 RoPE 缩放方法仍包含大量经验性设计，例如基频、频带边界、注意力温度以及长短序列缩放因子。目前还缺少一套理论，能够根据模型维度、训练长度和数据特征直接推导最优配置。未来值得研究的方向包括：分析模型能够识别哪些位置频率；设计可学习且带约束的位置变换；让不同层和不同注意力头使用不同尺度；以及在扩展上下文时尽量保持原模型函数不变的参数映射方法。

11.3 长文推理与检索能力不是同一能力

能够找到证据，并不等于能够正确整合证据。模型可能可以复述埋在长文中的 needle，却无法完成跨越 100K token 的变量绑定、时间顺序判断或全局约束推理。训练目标需要显式鼓励证据关系构建、来源引用、状态更新和不确定性表达；评测设计也必须排除只依赖局部信息就能答题的捷径。

11.4 平方复杂度与信息价值不匹配

在超长序列中，绝大多数 token 对之间都没有必要进行稠密交互。未来的架构可能会结合可学习的稀疏路由、分层摘要、外部记忆、状态空间模型和检索模块，使计算量更多地取决于“有效依赖”的数量，而不是原始序列长度。难点在于，路由机制既要保持高召回率，又要便于训练和解释，还必须在真实硬件上带来速度提升，而不能只停留在理论 FLOPs 的下降。

11.5 KV 缓存将成为服务资源与记忆治理问题

百万上下文的 KV 缓存不仅会占用大量显存，还涉及跨轮次复用、不同用户之间的隔离、缓存淘汰、压缩和隐私保护。未来需要回答：哪些 token 的 KV 可以量化、合并或丢弃；如何根据具体任务控制压缩误差；如何安全缓存含有敏感信息的前缀。GQA、MLA、低比特 KV、分层存储和语义压缩很可能进一步结合。

11.6 数据稀缺、污染和长尾结构

真正包含 100K 以上有效依赖的高质量文档并不多。反复拼接短文或大量使用自动生成数据，容易制造并不存在的长距离依赖，也可能污染评测集。未来需要开发结果可验证的合成数据生成器，采用基于文档关系图的采样方法，按代码仓库或时间进行严格拆分，并在数据卡中报告训练数据的“依赖距离分布”。LongBench 的双语、多任务设计 [18] 也说明，长上下文研究不能只关注英文检索。

11.7 多模态与 agent 上下文需要新的位置语义

视频帧、音频采样、网页操作和工具调用同时具有时间、空间和层级结构；如果只使用一维 token 位置，不同类型的语义距离就会混在一起。未来的位置表示可能需要采用多轴结构，并结合模态时间戳、文档树、代码调用图以及智能体事件之间的因果关系。长期运行的智能体还需要回答“哪些信息应当写入记忆、哪些信息应当遗忘、记忆如何审计”，而不能只是不断扩大 prompt。

11.8 可持续性与经济性

长窗口会显著增加预填充阶段的能耗和首 token 延迟。今后的研究应同时报告模型质量、GPU 小时、能耗、峰值显存以及每百万 token 的服务成本，并从质量与成本的 Pareto 前沿比较稠密长上下文、稀疏模型、RAG 和分层摘要。真正值得优化的指标不是最大的标称长度，而是在给定成本下能够可靠利用的上下文长度。

12 结论

LLM 的长上下文扩展并不是某一种算法的单独升级，而是位置、能力和系统三类瓶颈依次显现的过程。首先遇到的是位置编码能否稳定外推，其次是模型能否真正学会长距离依赖，最后则是注意力计算、KV 缓存和跨设备通信能否承担相应开销。从 8K 扩展到 32K，通常可以主要依靠 RoPE 缩放和轻量训练；从 32K 扩展到 256K，需要频率感知方法、长文本数据和显存友好的注意力协同工作；从 256K 进一步扩展到 1M，则需要非均匀且渐进的位置扩展，以及面向超长序列的分布式系统。2025 年以来的 NSA、DSA 和 Kimi Linear 进一步表明，下一阶段的竞争重点正在由“如何扩展稠密注意力”转向“如何从预训练阶段就建立稀疏或状态化的长程记忆”。

工程上最重要的原则，是分别验收“接口能够接收”“数值保持稳定”“模型能够检索”“模型能够推理”和“成本可以接受”这五个层次。只有位置编码、训练数据、系统实现和能力评测四个环节同时达到要求，百万 token 才能算作真正可用的模型能力，而不只是接口上的一个参数。

参考文献

- [1] A. Vaswani et al. *Attention Is All You Need*. NeurIPS, 2017. <https://arxiv.org/abs/1706.03762>
- [2] J. Su et al. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. 2021. <https://arxiv.org/abs/2104.09864>
- [3] O. Press, N. A. Smith, M. Lewis. *Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation*. ICLR, 2022. <https://arxiv.org/abs/2108.12409>
- [4] S. Chen et al. *Extending Context Window of Large Language Models via Positional Interpolation*. 2023. <https://arxiv.org/abs/2306.15595>
- [5] B. Peng et al. *YaRN: Efficient Context Window Extension of Large Language Models*. ICLR, 2024. <https://arxiv.org/abs/2309.00071>

- [6] Y. Ding et al. *LongRoPE: Extending LLM Context Window Beyond 2 Million Tokens*. ICML, 2024. <https://arxiv.org/abs/2402.13753>
- [7] T. Dao et al. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS, 2022. <https://arxiv.org/abs/2205.14135>
- [8] H. Liu, M. Zaharia, P. Abbeel. *Ring Attention with Blockwise Transformers for Near-Infinite Context*. 2023. <https://arxiv.org/abs/2310.01889>
- [9] Hugging Face. *Utilities for Rotary Embedding*. Transformers Documentation. https://huggingface.co/docs/transformers/internal/rope_utils
- [10] D. Narayanan et al. *Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM*. SC, 2021. <https://arxiv.org/abs/2104.04473>
- [11] NVIDIA. *Megatron Core Context Parallel Package*. Developer Guide. https://docs.nvidia.com/megatron-core/developer-guide/latest/user-guide/features/context_parallel.html
- [12] Y. Fu et al. *Data Engineering for Scaling Language Models to 128K Context*. ICML, 2024. <https://arxiv.org/abs/2402.10171>
- [13] Y. Bai et al. *LongAlign: A Recipe for Long Context Alignment of Large Language Models*. ACL, 2024. <https://arxiv.org/abs/2401.18058>
- [14] A. Grattafiori et al. *The Llama 3 Herd of Models*. 2024. <https://arxiv.org/abs/2407.21783>
- [15] Gemini Team. *Gemini 1.5: Unlocking Multimodal Understanding across Millions of Tokens of Context*. 2024. <https://arxiv.org/abs/2403.05530>
- [16] A. Yang et al. *Qwen2.5-1M Technical Report*. 2025. <https://arxiv.org/abs/2501.15383>
- [17] C.-P. Hsieh et al. *RULER: What's the Real Context Size of Your Long-Context Language Models?*. COLM, 2024. <https://arxiv.org/abs/2404.06654>
- [18] Y. Bai et al. *LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding*. ACL, 2024. <https://arxiv.org/abs/2308.14508>
- [19] J. Yuan et al. *Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention*. 2025. <https://arxiv.org/abs/2502.11089>
- [20] Kimi Team. *Kimi Linear: An Expressive, Efficient Attention Architecture*. 2025. <https://arxiv.org/abs/2510.26692>
- [21] Y. Bai et al. *LongBench v2: Towards Deeper Understanding and Reasoning on Realistic Long-context Multitasks*. 2024. <https://arxiv.org/abs/2412.15204>
- [22] DeepSeek-AI. *DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models*. 2025. <https://arxiv.org/abs/2512.02556>
- [23] Kimi Team. *Kimi K3: Open Frontier Intelligence*. 2026. <https://arxiv.org/abs/2607.24653>
- [24] Z. Dai et al. *Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context*. ACL, 2019. <https://arxiv.org/abs/1901.02860>
- [25] I. Beltagy, M. E. Peters, A. Cohan. *Longformer: The Long-Document Transformer*. 2020. <https://arxiv.org/abs/2004.05150>
- [26] M. Zaheer et al. *Big Bird: Transformers for Longer Sequences*. NeurIPS, 2020. <https://arxiv.org/abs/2007.14062>
- [27] T. Dao. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*. 2023. <https://arxiv.org/abs/2307.08691>
- [28] J. Ainslie et al. *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*. EMNLP, 2023. <https://arxiv.org/abs/2305.13245>
- [29] J. Ding et al. *LongNet: Scaling Transformers to 1,000,000,000 Tokens*. 2023. <https://arxiv.org/abs/2307.02486>
- [30] Y. Chen et al. *LongLoRA: Efficient Fine-tuning of Long-Context Large Language Models*. ICLR, 2024. <https://arxiv.org/abs/2309.12307>

- [31] W. Kwon et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP, 2023. <https://arxiv.org/abs/2309.06180>
- [32] G. Xiao et al. *Efficient Streaming Language Models with Attention Sinks*. ICLR, 2024. <https://arxiv.org/abs/2309.17453>
- [33] N. F. Liu et al. *Lost in the Middle: How Language Models Use Long Contexts*. TACL, 2024. <https://arxiv.org/abs/2307.03172>
- [34] L. Zheng et al. *SGLang: Efficient Execution of Structured Language Model Programs*. NeurIPS, 2024. <https://arxiv.org/abs/2312.07104>
- [35] X. Zhang et al. ∞ Bench: Extending Long Context Evaluation Beyond 100K Tokens. ACL, 2024. <https://arxiv.org/abs/2402.13718>
- [36] Q. Qin et al. *Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving*. FAST, 2025. <https://arxiv.org/abs/2407.00079>